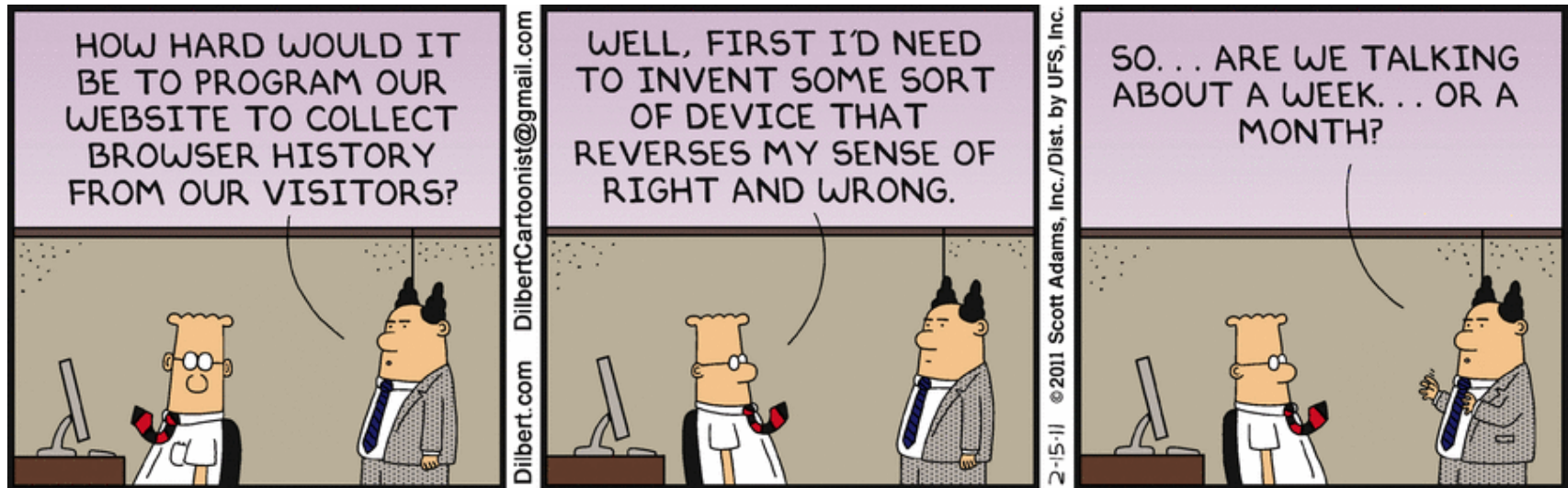


Lecture 26

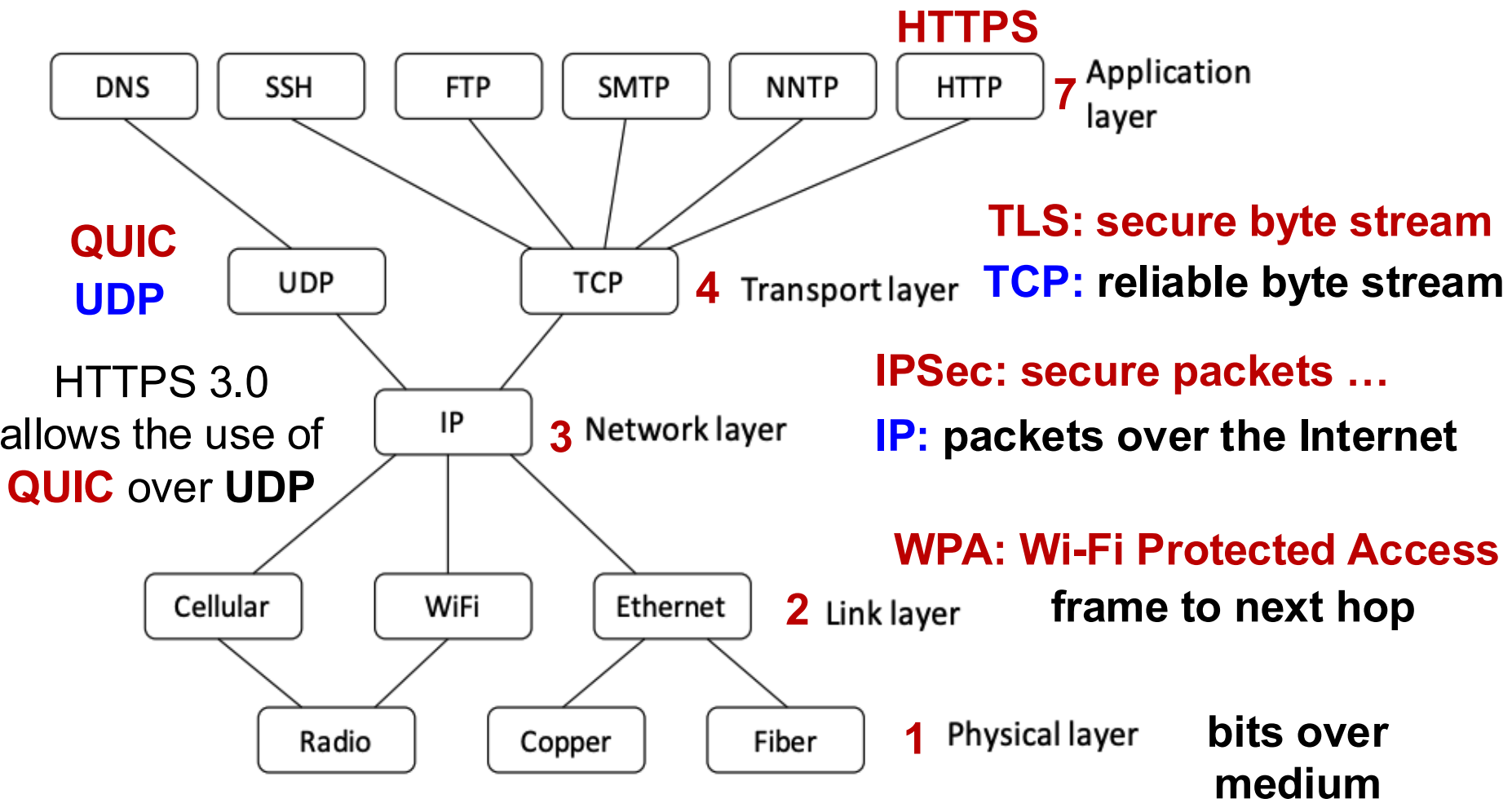
Web Security



Recap: The Internet Protocol Suite



Secured and **unsecured** communication/protocols in every layer



What is the Web?



The Internet is the global network of networks that uses the Internet protocol suite to communicate

The **World Wide Web (WWW)** is the global collection of documents (and other resources) linked by hyperlinks and accessed using HTTP or HTTPS
HTTP/HTTPS are application layer protocols and use TCP/IP

⇒ **WWW is an internet-linked collection of web resources**

A **web page** is a hypertext document (text and other media) written in **HTML**
Hypertext Markup Language 

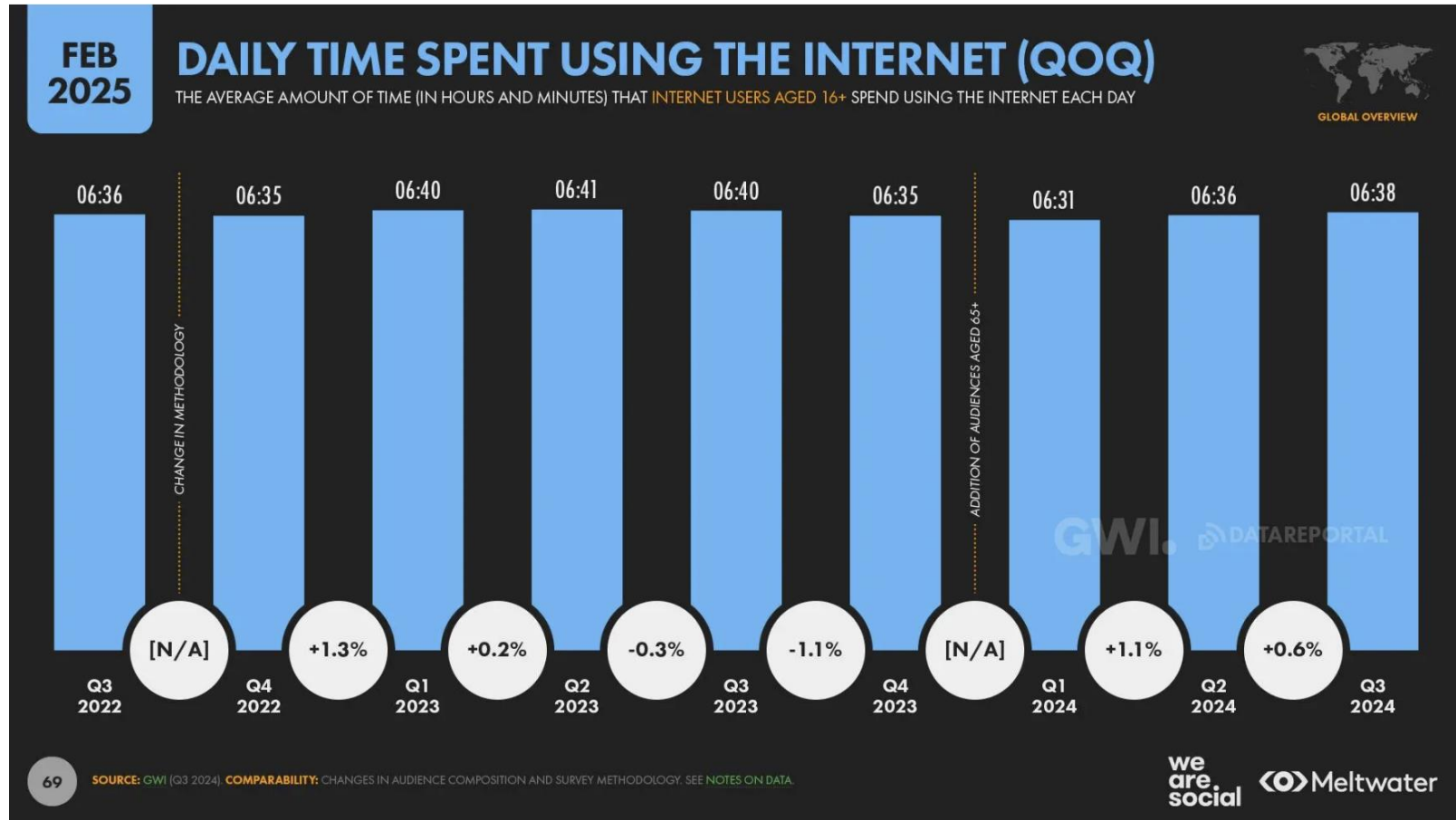
A **website** is a collection of multiple web pages and other web resources with a common **domain name** stored on one or more **web servers**

A **web browser** enables access to web resources – it makes requests to web servers using HTTP/S to view/execute the desired resources

Time Spent Using the Internet



A big chunk of our lives is spent using web resources over the Internet



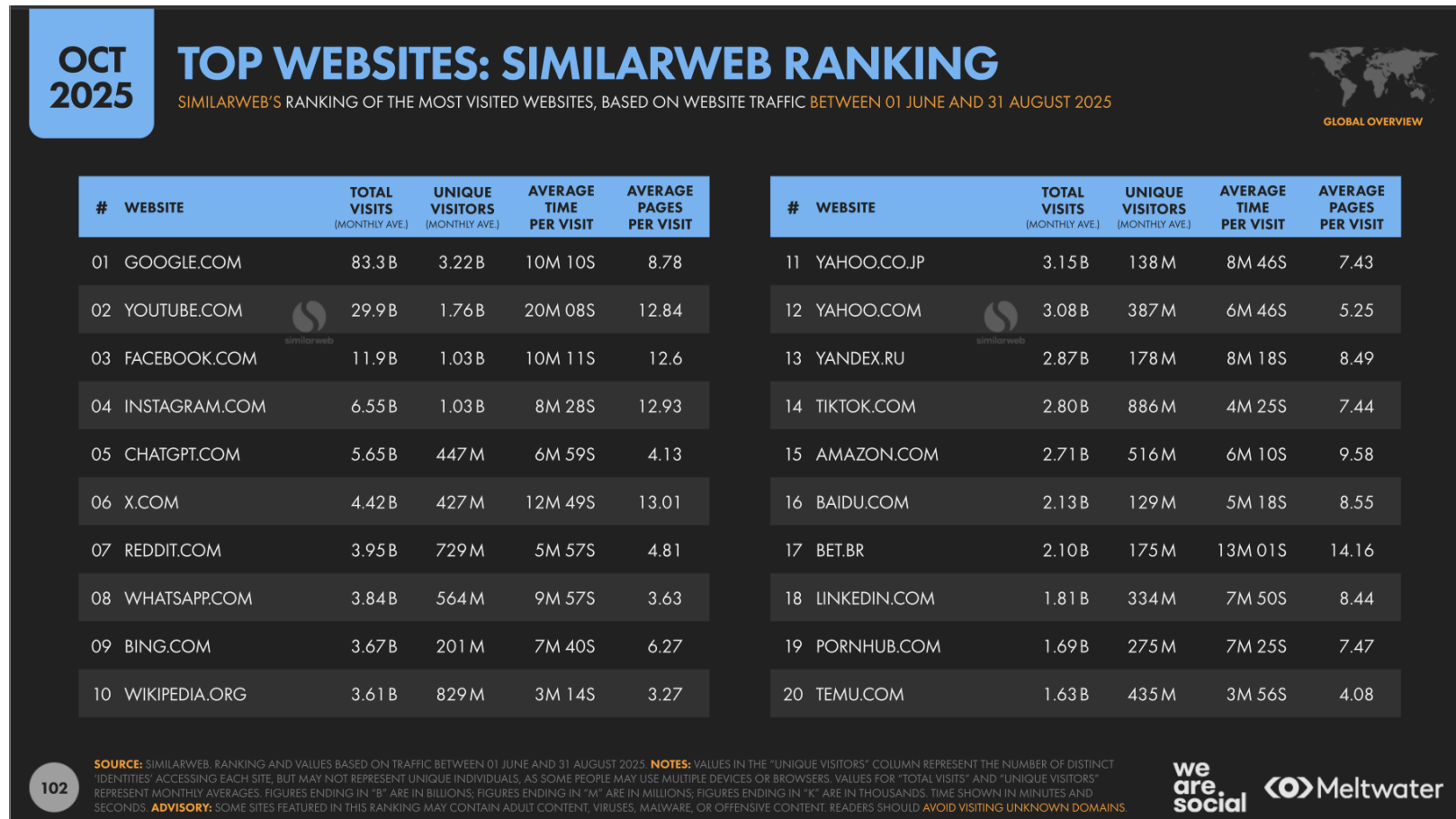
Source: <https://www.meltwater.com/en/blog/digital-2025>

Latest version no longer reports average time spent using the Internet

Time Spent Using the Internet



Where do we waste spend our time?

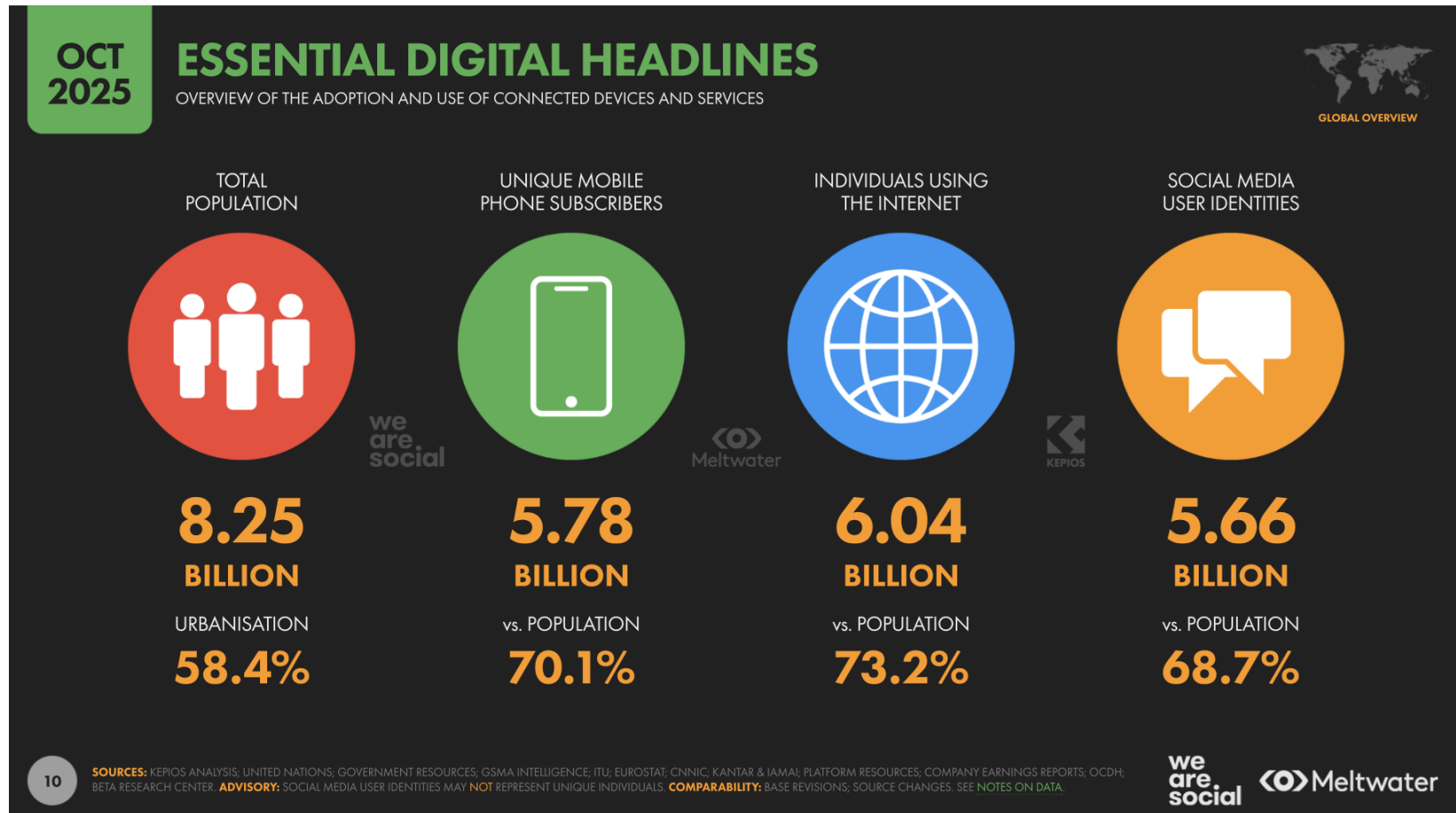


For more: <https://wearesocial.com/us/blog/2025/10/digital-2026-global-overview-report/>

Why Web Security?



A big chunk of our lives is spent using web resources over the Internet



3/4 of the human population is connected

Web Security



What is **web security**?

The ability to **safely browse the web**

Visit **any** website – including malicious ones! – without incurring any harm

- A website cannot access/steal data, install malware, access the camera or microphone on your computer, mobile device etc.
- A website cannot access/affect your session with another website

The ability to **run secure web apps**

- Web-based applications such as online meetings, games, file-sharing etc.

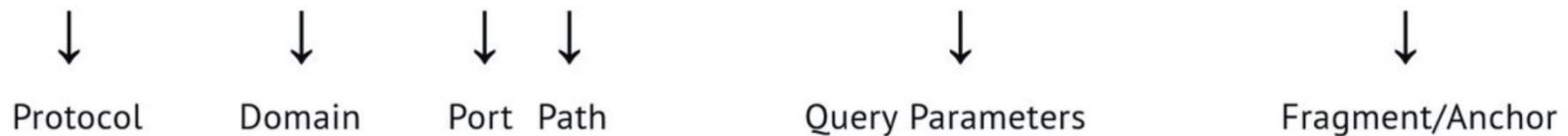
HTTP – Hypertext Transfer Protocol



The **Hypertext Transfer Protocol (HTTP)** is an application layer protocol for fetching web resources (e.g., an HTML document) from a web server

Every web resource has a **Uniform Resource Location (URL)**

`https://example.com:80/blog?search=test&sort_by=created_at#header`



HTTP is a simple protocol that allows to actions: **Request + Response**

- Requests are initiated by the client: the web browser
- Servers respond with the requested resource

HTTP Request

GET /index.html HTTP/1.1

HTTP Response

HTTP/1.0 200 OK

Content-Type: text/html

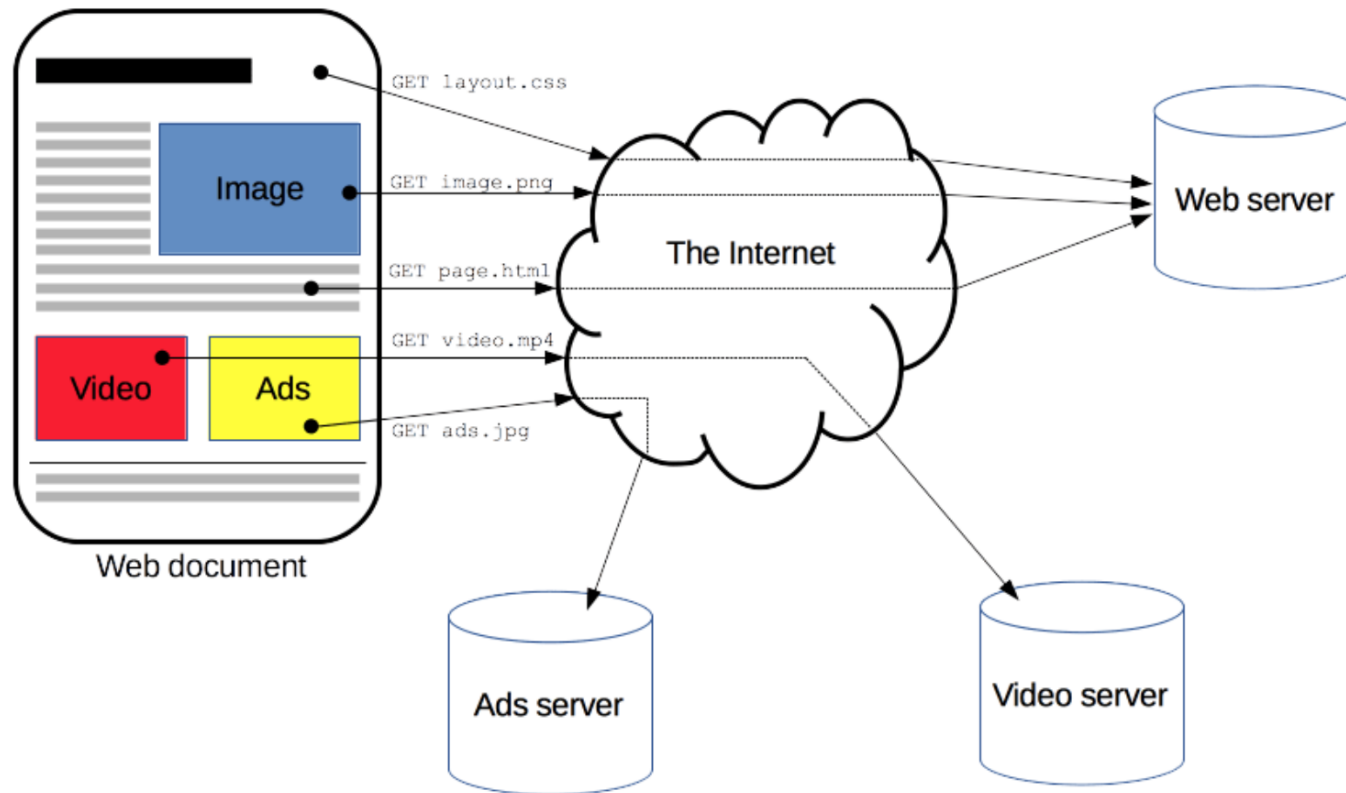
<html>Some data... </html>

HTTP – Hypertext Transfer Protocol



A complete document is reconstructed from different sub-documents: text, layout description, images, videos, scripts, etc.

which can be fetched from different **web servers** $\Rightarrow$ **Larger attack surface**



One Website



dispatch.com

Local Weather
Local Script

SUBSCRIBE NOW
\$1 for the first month

MARKETPLACE
Search For & Place Classifieds

The Columbus Dispatch

News Sports Business Opinion Real Estate Advertise Obituaries eNewspaper Legals

43°F

Subscribe Sign In



What a week at OSU as president resigned, replaced in 6 days

On the eve of a board meeting where the OSU board of trustees is expected to name a new leader, here's a timeline of Carter's resignation.

EDUCATION 9:36 a.m. ET March 13



OSU looks inward for leadership amid turmoil of another failed presidency

Ohio State's decision to skip a national search is nearly unheard of for a research university of its size.

EDUCATION 9:34 a.m. ET March 13



Gas prices are up. Here's how to save | Getting There

TRANSPORTATION 6:02 a.m. ET March 13

Is it legal? Can Ohio kids stay home alone during spring break?

At what age can kids stay home alone during spring break? Here's what laws in Ohio have to say.

FAMILY 11:18 a.m. ET Mar. 12



Top Headlines

Olentangy Schools confirms broader investigation into Liberty wrestling coach

1 dead, 1 injured in Fairfield County head-on crash

Trump Delusion Syndrome is scariest malady of all

LETTERS Letters to the Editor

New Ohio State president draws praise and pushback from campus leaders

Blue Jackets stonewalled by their former star goalie again

84-year-old woman killed in crash with dump truck that lost tire

Columbus police to auction impounded vehicles this weekend

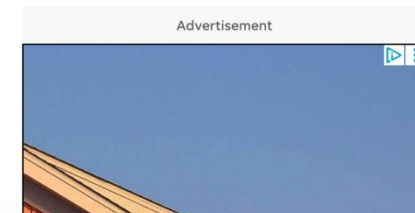
Check your spam folder. Beatty missed invite for Kennedy Center meeting

Get top stories in your inbox

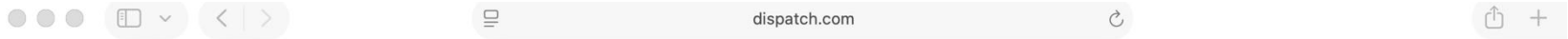
Video



Framed Ad



One Website



MARKETPLACE

Search For & Place Classifieds

The Columbus Dispatch

SUBSCRIBE NOW

\$1 for the first month

News Sports Business Opinion Real Estate Advertise Obituaries eNewspaper Legals 43°F [Subscribe](#) [Sign In](#)

Extra, Extra



ENTER NOW! ENTER OFTEN! →

You could win a family membership or tickets to The Science Behind PIXAR!



Enter Today! →

Enter for a chance to WIN a FREE week of Columbus Academy Summer Experience



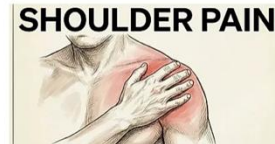
Enter Now! →

You could win a Blue Jackets prize package including a trip to the ice!

[More in Contests →](#)



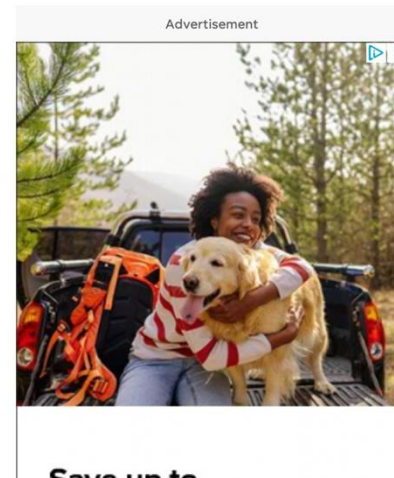
Here's The Estimated Walk-In



Shoulder Pain is Not From



The Spring Garden Upgrade



Save up to

Ad

More Videos

**More
3rd
Party
Ads**

Analytics

One Website



One website? – Think again

```
<title>The <link rel=dns-prefetch href="//hopenbid.pubmatic.com/>
<meta prope <link rel=preconnect href=https://hopenbid.pubmatic.com/>
<link rel=d <link rel=dns-prefetch href="//ads.pubmatic.com/>
<link rel=p <link rel=dns-prefetch href="//ib.adnxs.com/>
<link rel=d <link rel=preconnect href=https://ib.adnxs.com/>
<link rel=d <link rel=dns-prefetch href="//grid.bidswitch.net/>
<link rel=d <link rel=preconnect href=https://grid.bidswitch.net/>
<link rel=p <link rel=dns-prefetch href="//ads.yieldmo.com/>
<link rel=d <link rel=preconnect href=https://ads.yieldmo.com/>
<link rel=p <link rel=dns-prefetch href="//tlx.3lift.com/>
<link rel=d <link rel=preconnect href=https://tlx.3lift.com/>
<link rel=p <link rel=dns-prefetch href="//e.serverbid.com/>
<link rel=p <link rel=preconnect href=https://e.serverbid.com/>
<link rel=d <link rel=dns-prefetch href="//a.teads.tv/>
<link rel=d <link rel=preconnect href=https://a.teads.tv/>
<link rel=d <link rel=dns-prefetch href="//search.spotxchange.com/>
<link rel=d <link rel=preconnect href=https://search.spotxchange.com/>
<link rel=p <link rel=dns-prefetch href="//prebid-server.rubiconproject.com/>
<link rel=d <link rel=preconnect href=https://prebid-server.rubiconproject.com/>
<link rel=p <link rel=dns-prefetch href="//confiant-integrations.global.ssl.fastly.net/>
<link rel=d <link rel=preconnect href=https://confiant-integrations.global.ssl.fastly.net/>
<link rel=p <link rel=dns-prefetch href="//polarcdn-terrax.com/>
<link rel=d <link rel=preconnect href=https://polarcdn-terrax.com crossorigin/>
<link rel=d <link rel=preconnect href=https://polarcdn-terrax.com/>
<link rel=p <link rel=dns-prefetch href="//bw-prod.plrsrvcs.com/>
<link rel=d <link rel=dns-prefetch href="//polarcdn-pentos.com/>
<link rel=p <link rel=dns-prefetch href="//geolocation.onetrust.com/>
<link rel=d <link rel=preconnect href=https://geolocation.onetrust.com/>
<link rel=d <link rel=dns-prefetch href="//cdn.taboola.com/>
<link rel=d <link rel=preconnect href=https://cdn.taboola.com/>
<link rel=d <link rel=dns-prefetch href="//trc.taboola.com/>
<link rel=d <link rel=preconnect href=https://trc.taboola.com/>
<link rel=p <link rel=dns-prefetch href="//www.usatodaynetworkservice.com/>
<link rel=d <link rel=preload as=fetch type=application/json href=https://user.dispatch.com/NCOD-GUP/user/ crossorigin=use-credentials/>
<link rel=p <link rel=preload as=script href=/tangstatic/js/pbjssandwich-qla2z35eac4b1d.min.js/>
<link rel=d <link rel=preload as=script href=https://c.amazon-adsystem.com/aax2/apstag.js/>
<link rel=p <link rel=preload as=script href=https://static.adsafeprotected.com/iasPET.1.js/>
<link rel=d <link rel=preload as=script href=https://securepubads.g.doubleclick.net/tag/js/gpt.js/>
<link rel=p <link rel=preload as=script href=https://confiant-integrations.global.ssl.fastly.net/r5TdgVvkbv-PeaJCKaQfCh5Xsto/gpt_and_prebid
<link rel=d <link rel=preload as=script href=/tangstatic/js/main-qla2z36e434966.min.js/>
<link rel=p <link rel=preload as=script href=https://geolocation.onetrust.com/cookieconsentpub/v1/geo/location/gnt.otgc/>
<link rel=d <link rel=preload as=script href=https://cdn.taboola.com/libtrc/gannett-network/loader.js/>
<link rel=p <meta name=description content="The Columbus Dispatch is the number one source for Columbus and Ohio breaking politics, busines
s."/>
```


JavaScript



JavaScript is a programming language that adds interactivity to a website. Originally, HTML content was static – the server generated the content and returned it to the web browser to be displayed.

Today, websites also deliver scripts to be run inside of the browser – e.g., with games, animation, forms, buttons etc.

```
<button onclick="alert('The date is' + Date())">  
    Click me to display Date and Time.  
</button>
```

JavaScript can

- make additional web requests
- manipulate page
- read browser data
- read input from local hardware, e.g., `onClick`, `onMouseover` etc.

exceptionally powerful

⇒ **Even larger attack surface**

HTTP or Browser Cookies



HTTP is **stateless** – each request is handled independently of past activity

If HTTP is stateless, how do we have website sessions?



A **cookie** is a small piece of data that a server sends to the web browser
The browser may store and send it back in future requests to that server

Cookies are used for three purposes

Session management

- Logins, shopping carts, game scores, or any other session state

Personalization

- User preferences, themes, and other settings

Tracking

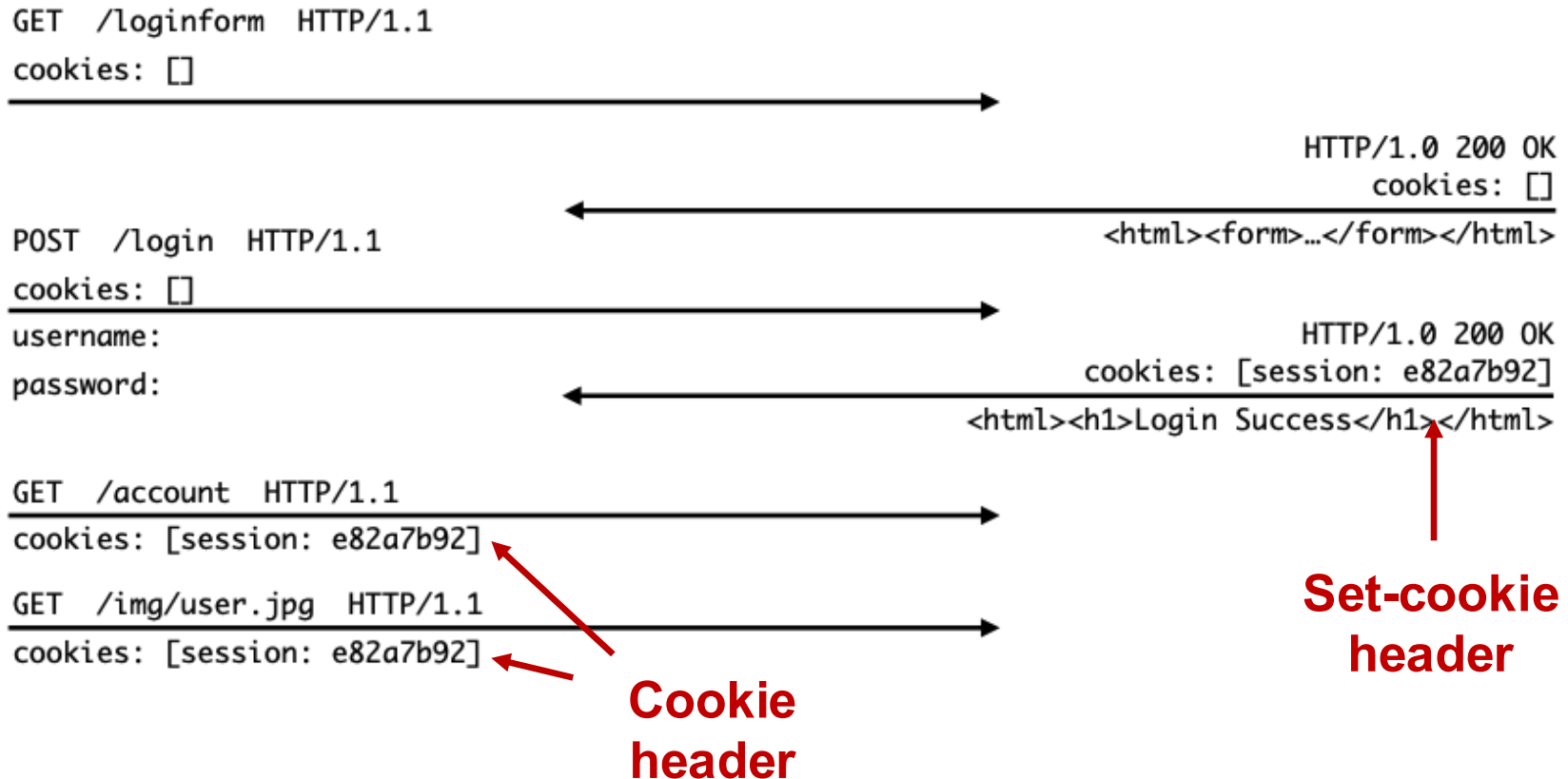
- Recording and analyzing user behavior ⇒ **Privacy concerns**

Session Management using Cookies



Web server sends **set-cookie header** with its response

Browser sets cookie and sends it with subsequent requests to the server inside the **cookie header** – until the cookie expires



Session Hijacking Attack



Attacker **steals the cookie of an existing session** and interacts with the server as if it were the authenticated user

e.g., when only the login page is encrypted and not all session, through packet sniffing, malicious JavaScript, etc.

GET /loginform HTTP/1.1

cookies: []

HTTP/1.0 200 OK

cookies: []

POST /login HTTP/1.1

cookies: []

username:

password:

<html><form>...</form></html>

HTTP/1.0 200 OK

cookies: [session: e82a7b92]

<html><h1>Login Success</h1></html>

GET /account HTTP/1.1

cookies: [session: e82a7b92]

GET /img/user.jpg HTTP/1.1

cookies: [session: e82a7b92]

Critical to restrict access to cookies to prevent **cookie poisoning attacks**

Restricting Access to Cookies



Two ways to ensure that cookies are **are not accessed by third parties**

- the **Secure** attribute
- the **HttpOnly** attribute

Cookies with the Secure attribute

- can be set and sent **only** when using the HTTPS protocol
- recall: HTTPS = HTTP over TLS: encryption and server authentication
- attackers cannot easily access such cookies – e.g., no man-in-the-middle
- **But nothing is 100% secure**: attackers could access client's hard disk
- JavaScript can read and modify the information stored in secure cookies
- **unless** the **HttpOnly** attribute is set as well

Cookies with the HttpOnly attribute are inaccessible to JavaScript

BUT all cookies can be read and modified by the end user

⇒ **Cookie Poisoning** where malicious user can become a threat for server

Restricting Access to Cookies



Restricting temporal access – i.e., **how long should a cookie live?**

- **Session cookies** are deleted when the current session ends as defined by the browser
- they can last indefinitely if the browser restores *sessions* when restarting
- **Permanent cookies** are deleted at a specified time using the `Expires` or `Max-Age` attributes

Cookies with critical security implications should be short-lived

Where should a cookie be sent?

`Domain` and `Path` attributes define what URLs the cookies should be sent to

When the `Domain` attribute is set – cookie is sent to all subdomains

When the `Path` attribute is set – cookie is sent to all all subdirectories

Alternatively, `SameSite` attribute can control where a cookie should be sent

Same-Origin Policy (SOP)



Not just cookies – recall web security goal: A website should not be able to access/affect your session on another website

Same-Origin Policy: Resources (document, script etc.) from one **origin** cannot access (read/modify) resources from **another origin**

Same origin = same scheme/protocol, host, and port (if specified)

`http://example.com/dir/one.html`



`http://example.com/dir2/two.html`



`https://example.com/dir/one.html`



`https://a.example.com/dir/one.html`



`http://attacker.com/dir/one.html`



SOP protects confidentiality and integrity of information from different origins even if they are loaded on the same webpage

Crossing the Same-Origin Policy



Same-Origin Policy: JavaScript from one origin can only read/modify data from the *same* origin

Critical – Otherwise, JavaScript from *attacker.com* could easily access your cookies from *bank.com*

When JavaScript is loaded through HTML inside a `<script>` tag
Browser knows if JavaScript comes from *attacker.com* or *bank.com*

What if the attacker finds a way to inject JavaScript into *bank.com*?
JS from attacker can access all data from *bank.com* – including cookies
The same-origin policy is crossed

⇒ **Cross Site Scripting (XSS) Attack**

XSS High-Level Idea



A normal search request looks like this

<https://google.com/search?q=apple>

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for <?php echo $_GET["q"] ?></h1>
  </body>
</html>
```

Sent to Browser

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for apple</h1>
  </body>
</html>
```

XSS High-Level Idea



To embed a script – simply search for it

`https://google.com/search?q=<script>alert("hello")</script>`

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for <?php echo $_GET["q"] ?></h1>
  </body>
</html>
```

Sent to Browser

```
<html>
  <title>Search Results</title>
  <body>
    <h1>Results for <script>alert("hello")</script></h1>
  </body>
</html>
```

Cross Site Scripting (XSS)



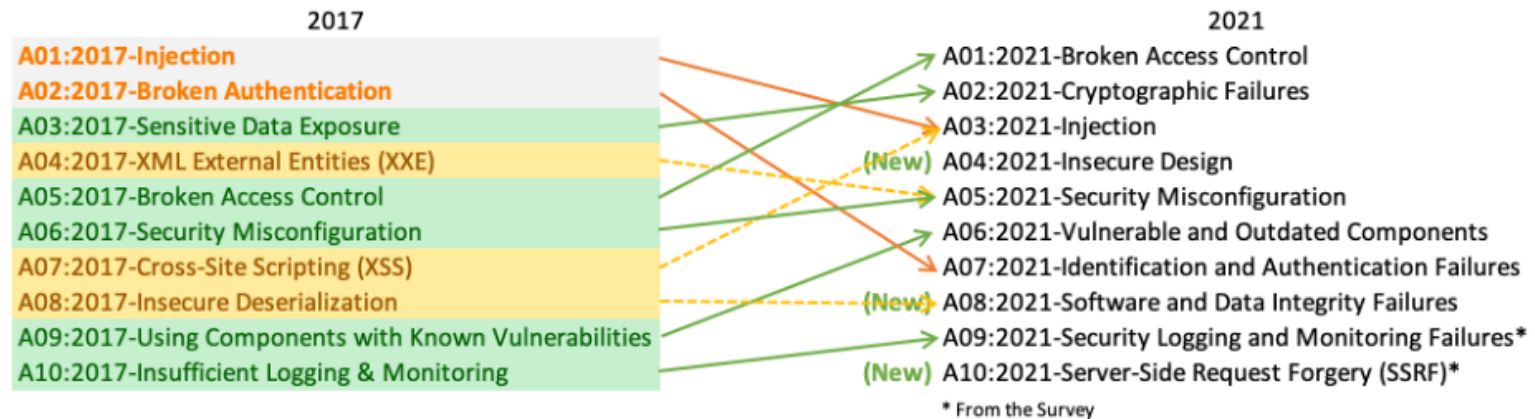
In a **Cross Site Scripting (XSS) Attack** an attacker injects scripting code (i.e. JavaScript) into pages generated by a web application

XSS attack occurs when an application takes **untrusted data** and sends it to a web browser **without proper validation** or **sanitization**

As a result, attacker's malicious code is executed on victim's browser

XSS is in fact an **injection attack** – attacker manages to inject malicious JavaScript into an origin's response

Classified as such in new OWASP Top10 Application Security Issues



Injection Attacks in OWASP Top 10



OWASP Top10 Application Security Issues 2025

Top 10:2025 List

<https://owasp.org/Top10/2025/>

1. [A01:2025 - Broken Access Control](#)
2. [A02:2025 - Security Misconfiguration](#)
3. [A03:2025 - Software Supply Chain Failures](#)
4. [A04:2025 - Cryptographic Failures](#)
5. [A05:2025 - Injection](#)
6. [A06:2025 - Insecure Design](#)
7. [A07:2025 - Authentication Failures](#)
8. [A08:2025 - Software or Data Integrity Failures](#)
9. [A09:2025 - Security Logging and Alerting Failures](#)
10. [A10:2025 - Mishandling of Exceptional Conditions](#)

XSS in CVE Database



<https://www.cve.org/CVERecord/SearchResults?query=xss>

The screenshot shows the CVE.org website interface. At the top, there is a navigation bar with links: About, Partner Information, Program Organization, Downloads, Resources & Support, and Report/Request. Below this is a search bar with the text 'xss' entered, a 'Search' button, and a 'Site Search' button. A notice banner states: 'Notice: Expanded keyword searching of CVE Records (with limitations) is now available in the search box above. Learn more here.' The main section is titled 'Search Results' and shows 'Showing 1 - 25 of 40,033 results for xss'. There are filters for 'Show: 25' and 'Sort by: CVE ID (new to old)'. Two results are displayed:

- CVE-2026-3962** CNA: VulDB
A vulnerability was identified in Jcharis Machine-Learning-Web-Apps up to a6996b634d98ccec4701ac8934016e8175b60eb5. The impacted element is the function render_template of the file Machine-Learning-Web-Apps-master/Build-n-Deploy-Flask-App-with-Waypoint/app/app.py of the component Jinja2 Template Handler. Such manipulation leads...
[Show more](#)
- CVE-2026-3951** CNA: VulDB
A security flaw has been discovered in LockerProject Locker 0.0.0/0.0.1/0.1.0. Affected is the function authIsAwesome of the file source-code/Locker-master/Ops/registry.js of the component Error Response Handler. The manipulation of the argument...
[Show more](#)

Defense Against XSS



Filtering or input validation and sanitization

- First line of defense – necessary but not sufficient
- Validate all headers, cookies, query strings, form fields, and hidden fields (i.e., all parameters) against a rigorous specification of what is allowed
- Can look for attack signatures – but difficult to update and lists likely to be incomplete
- Many ways to fool filters – search “XSS filter evasion”
- Different ways to call JavaScript and encode content

Cookie Security

- Use `HttpOnly` cookies, `SameSite=strict` cookies

Disabling all JavaScript

Content Security Policy (CSP)

- Set of policies based on trust that add another layer of security to browsers

SQL Injection



SQL (Structured Query Language) Injection is another injection attack – this time attacker's malicious code is executed on the web server database

Attackers can use SQL injection on an application if

- it has dynamic database queries
- that use string concatenation and user supplied input
- fail to validate user input

e.g.

```
String query = "SELECT account_balance FROM user_data WHERE user_name = "
               + request.getParameter("customerName");

try {
    Statement statement = connection.createStatement( ... );
    ResultSet results = statement.executeQuery( query );
}
```

Attacker enters an SQL command as customerName

SQL Injection Defenses



Attackers can use SQL injection on an application if

- it has dynamic database queries
- that use string concatenation and user supplied input
- fail to validate user input

Prevent any – preferably all - of above points from happening

e.g., validate all user input: **Allow-list, Input Validation**

